

Project 5: Federated and Distributed Learning

TA: [Riccardo Zaccone](mailto:riccardo.zaccone@polito.it) (riccardo.zaccone@polito.it)

Overview

Modern success of deep learning and the breakthroughs it enables rely on increasingly complex models, whose training requires growing availability of large amounts of data and computing resources. State-of-art architectures, especially those related to NLP, require billions or trillions of parameters, leading to real challenges in training and serving such solutions for real-world applications. In fact, the massive scale of these models leads to prohibitively long training times and makes it impossible to use them on a single general purpose device such as a GPU. Moreover, as hardware speedup scaling laws are already on point, next generation deep learning demands for overcoming the limits of current algorithms and architectures.

To tackle these challenges, distributed methods for deep learning of neural networks have been studied, mainly grouped into *model-parallel* and *data-parallel* methods: the former consists in ways of distributing the computations of a single model splitting its modules across different accelerators, while the latter consists in processing different chunks of data in parallel using multiple accelerators.

Centralized training solutions do not manifest their inadequacies only for learning at scale, but also for leveraging the decentralized nature of data as arising in real-world applications. In practice, the centralized training paradigm no longer matches the needs of modern applications, which need to learn from heterogeneous and massively decentralized data.

Distributed Learning

1. Goals

In this project we will restrict ourselves to studying data-parallelism approaches. The goal is to become familiar with the objectives of distributed learning of deep neural networks, understand the main challenges and practice with the main approaches in literature. Finally, you will have the opportunity to identify open problems and suggest solutions.

Become familiar with the literature and the state-of-the-art

Before starting the project, you should take some time to study and get familiar with distributed learning, its algorithms, challenges and the main proposed solutions. Here some references:

- **Large Batch Training** [17, 18]
- **Local SGD** [19, 20]
- **Optimizers for distributed learning** [21]

2. Codebase, Resources & Dataset:

To start your experimentation you will first need to implement your codebase. The first step will be to implement a centralized training baseline, to compare the performance of distributed techniques to the standard centralized procedure. Next, extend your code to simulate the parallel training of local methods (e.g. algorithm 1 of [19]) by *sequentially* training your workers: this does not change the results of the algorithms, but it is necessary to work with one GPU only.

For the experiments, we require you to use the **model architecture described in [11] (LeNet-5)**, on the **CIFAR-100 dataset**, which can be downloaded from torchvision. As per the computational resources, you will use the free version of Google Colab.

Tips and tricks

Good code is initially harder and slower to develop, but it soon repays the initial effort: remind that the code is taken into account as part of the project, and it must be delivered as an attachment to your report. Please adhere to the best coding practices: take into care modularization, avoid deep nesting, mind about readability and lastly use a version control system.

It is in your interest to correctly implement checkpointing and experiment logging. Colab will likely interrupt your experiments, so you must be ready to **recover an interrupted run from a certain point** in training when resources will be granted again. Not taking this into account will most likely result in (much) more time for carrying out the experiments.

3. Experimentation

Preliminaries

As you may have noticed, the CIFAR-100 dataset you downloaded from torchvision has not a validation split. Your first step will be to split the original dataset such to have a validation set, to be used for hyperparameter tuning. For distributed training, you will also need to have different splits of the training set, so implement a procedure to perform such splits.

Centralized baseline

Train your CNN using SGDM and AdamW optimizers, taking care of searching for the best hyperparameters. For both optimizers, perform 150 epochs and use the cosine annealing learning rate scheduler.

Report the plots of training loss, test loss, training accuracy and test accuracy.

Test Large Batch Optimizers

Replace the standard optimizers with their large batch counterparts [17,18]. The suggested way to do it is to subclass the `torch.optim.Optimizer` class and implement the code for the respective algorithms. It is possible to start from the implementations of SGD and AdamW available in torch. Then, in your codebase, allow the optimizer to be chosen among [**SGDM**, **AdamW**, **LARS**, **LAMB**] when launching the script (using command line arguments). When experimenting with large-batch optimizers, start from the best hyperparameters found for the base algorithms, and tune the learning rate following [18]. Compare the performance of such optimizers with the one obtained by simply applying the learning rate schedule of [18] to the base optimizers. How do the results change? Up to which batch size is it possible to scale without significantly losing test accuracy?

Switch to Local Methods

Perform an iid sharding of the training set in $K=\{2,4,8\}$ chunks, and train using the LocalSGD algorithm [19]. Explore performing multiple local steps $J=\{4,8,16,32,64\}$, scaling accordingly the number of iterations (total number of batches processed). For all the experiments, you want to perform as many iterations as in the centralized scenario when increasing the number of local steps or the number of workers (K). Which is the combination that works best? What are the challenges in increasing the number of workers and local steps?

Try other distributed optimizers

There are more advanced solutions that use two optimizers, one for the outer loop and one for the inner one [21]. Extend your experimentation to these solutions. Analyze your results: evaluate the computation and communication cost of each alternative you tried and discuss pros and cons of large batch optimizers and local methods.

4. Time for your personal contribution

Now that you have a clear understanding of the distributed learning scenario and state-of-art solutions, you can now explore a more specific problem. This part of your project is purposely weakly supervised, to let you come up with original ideas. Here we list some of the research directions we believe are most promising.

- **Asynchronous algorithms:** in the above experimentation we limited ourselves to synchronous algorithms. There are some works on asynchronous methods for distributed learning [23, 22, 25] you could start exploring to have an idea of the setting and state-of-art approaches.
- **Optimize for the number of steps in local training methods:** as you may have noticed, there is a tradeoff in increasing the number of local steps. You found the best combination for the task at hand by treating J as a hyperparameter, and tuned on a validation set. A very recent work [26] has delved into the choice of optimal learning rate for LocalSGD, which is based also on the number of local steps. Can you come up with a way to dynamically decide how many steps are to be taken during the training process?

Deliverables

For the exam, you should deliver:

1. A **report** of your experimentation: it must clearly contain the results of the required experiments, plus whatever is needed to substantiate your personal contribution. Later in the development of the project you can ask more specific feedback about what to include and how to do it.
2. The **codebase to reproduce your results**: it should contain a README.md file containing the instructions to run your code to reproduce specific parts of your experimentation. The code itself is not evaluated, but at the exam we may ask for clarifications about some code portions.

Report guidelines

The report must be written in LaTeX, using the template available [here](#). It should list all the authors (team members) with associated student ID, in whatever order, and must be long at most 8 pages (references included). Regarding the contents, the report should answer all the questions mentioned in the track description, explaining the motivation with empirical and/or theoretical findings, which must be reported in the report itself. The main part of your report should be devoted to your proposed extension, explaining the methodology (e.g. what you are doing differently from existing approaches to solve the problem you chose) and later the experimental results. Implementation details of the experiments should also be included (e.g. how data has been splitted, how hyperparameter search has been conducted, etc.). You can look at published research papers to have an idea of such a structure.

Common rules

During all the phases of experimentation, remember to always apply the best deep learning practices, such as train-validation-test splitting, performing multiple independent runs for each experiment, ecc.

Before engaging in phase 4, always discuss your plan with the TA.

FAQ

Q: Should I buy online computing resources for doing the project?

A: NO! We don't require you to have access to any computing resource but the free version of Google Colab.

Q: I have a GPU and I prefer using it instead of Google Colab, is it allowed?

A: Yes, if you prefer to use your own resources you sure can. However, we don't require you to have additional computing power than the free version of Google Colab.

Q: Can I avoid proposing and/or implementing some of the points of "Base experimentation"? Can I avoid the "personal contribution" part?

A: The steps detailed in the "Base experimentation" part are all required and represent the bare minimum for the project to be considered of sufficient level. While it is possible to have a

sufficient grade developing the first part only, we strongly encourage you to take the challenge of personal contribution, it is the fun part of the project.

Q: I have developed a solution for the “personal contribution” part, but it is not state of the art in terms of results, or it performs worse than expected. Will this be considered as a penalty for the exam?

A: No, we don’t require you to advance the state of the art. If your solution performs worse than an existing method, this will not affect the evaluation. We mostly take into account the process of developing your solutions, and marginally take into account how better it is with respect to existing approaches. Being able to discuss your method and results is much more important than having strong results.

Q: I read the suggested papers and I found a lot of math. Do I need to provide theoretical analysis of the method/analysis I propose as a personal contribution?

A: No, we don’t require you to perform any theoretical analysis, even if we expect you to know the theoretical framework you’re working in (e.g. in FL you should know what the *client drift* is). If you want, your personal contribution can be theoretical, but it must still be supported by properly designed experiments. You can always reach out to the TA for your specific case.

References

- [1] Hsu et al., Measuring the effects of non-identical data distribution for federated visual classification, 2019.
- [2] Reddi et al., Adaptive federated optimization. ICLR 2021.
- [3] Kairouz et al., Advances and open problems in federated learning, Found. Trends Mach. Learn, 2021
- [4] Li et al., Federated optimization in heterogeneous networks. PMLR, 2018
- [5] Karimireddy et al., Scaffold: Stochastic controlled averaging for federated learning, PLMR 2020
- [6] Kairouz et al., Advances and Open Problems in Federated Learning, Foundations and Trends in Machine Learning, 2021
- [7] Li et al., A Survey on Federated Learning Systems: Vision, Hype and Reality for Data Privacy and Protection, 2019
- [8] Chen et al., Privacy and Fairness in Federated Learning: On the Perspective of Tradeoff, ACM computing surveys 2023
- [9] Pfeiffer et al., Federated Learning for Computationally Constrained Heterogeneous Devices: A Survey, ACM computing surveys 2023
- [10] McMahan et al., Communication-Efficient Learning of Deep Networks from Decentralized Data, AISTATS 2017
- [11] Hsu et al., Federated Visual Classification with Real-World Data Distribution, ECCV 2020**
- [12] Qu et al., Rethinking Architecture Design for Tackling Data Heterogeneity in Federated Learning, CVPR 2022
- [13] Pieri et al., Handling Data Heterogeneity via Architectural Design for Federated Visual Recognition, NeurIPS 2023

- [14] Li et. al., Fedbn: Federated learning on non-iid features via local batch normalization. ICLR 2021
- [15] Caldas et al. "Leaf: A benchmark for federated settings." Workshop on Federated Learning for Data Privacy and Confidentiality 2019
- [16] McCandlish, An empirical model of large-batch training, arXiv 2018
- [17] You et. Al, **Large Batch Training of Convolutional Networks**, arXiv 2017
- [18] You et. Al, **Large Batch Optimization for Deep Learning: Training BERT in 76 minutes**, ICLR 2020
- [19] Stich et al., **Don't Use Large Mini-Batches, Use Local SGD**, ICLR 2020
- [20] Yu et. Al, **On the Linear Speedup Analysis of Communication Efficient Momentum SGD for Distributed Non-Convex Optimization**, ICML 2019
- [21] Wang et. Al, **SlowMo: Improving Communication-Efficient Distributed SGD with Slow Momentum**, ICLR 2020
- [22] Nabli et al., **A2CiD2: Accelerating Asynchronous Communication in Decentralized Deep Learning**, NeurIPS 2023
- [23] Chen et al., **Accelerating Gossip SGD with Periodic Global Averaging**, ICML 2021
- [24] Nemeth et al., **A Snapshot of the Frontiers of Client Selection in Federated Learning**, TMLR 2022
- [25] **SHAT: A Novel Asynchronous Training Algorithm That Provides Fast Model Convergence in Distributed Deep Learning**
- [26] Balles et al., **On the Choice of Learning Rate for Local SGD**, TMLR 2024
- [27] Wang et al., **Towards Personalized Federated Learning via Heterogeneous Model Reassembly**, NeurIPS 2023
- [28] Collins et al., **Exploiting Shared Representations for Personalized Federated Learning**, ICML 2021
- [29] Fallah et a., **Personalized Federated Learning with Theoretical Guarantees: A Model-Agnostic Meta-Learning Approach**, NeurIPS 2020
- [30] Marfog et al., **Personalized Federated Learning through Local Memorization**, ICML 2022
- [31] Pillutla et al., **Federated Learning with Partial Model Personalization**, ICML 2022
- [32] Shamsian et al., **Personalized Federated Learning using Hypernetworks**, ICML 2021